



CSE430: Software Testing

Course Project Report

Project Title: Hotel Management System

Programming Language: Java

Team Name: Bug Busters

Group Members:

Ratul Podder — ID: 2022-1-60-067

Erin Dewan Oishe — ID: 2022-1-60-073

Faria Azad Anita — ID: 2022-1-60-252

Department of Computer Science and Engineering
Spring26

Submitted to:

Shartaz Sajid Nahid

Lecturer

Department of Computer Science & Engineering

Date of Submission: 18-04-2026

Table of Contents

1. Project Description	3
1.1 System Overview	3
1.2 Project Structure: Classes and Methods	3
2. Identification of the Testing Scope	5
2.1 Scope of Testing	5
2.2 Testing Perspectives per Method	5
3. Test Case Design	8
3.1 isValidName(): Validator	8
3.2 isValidPhone(): Validator	9
3.3 isValidGender(): Validator	10
3.4 Singleroom Constructor	11
3.5 Singleroom.getFoodBill()	12
3.6 Doubleroom Constructor	12
3.7 Doubleroom.getFoodBill()	13
3.8 Bill.calculate()	13
3.9 Hotel.getIndex()	15
3.10 Hotel.countAvailable()	16
3.11 holder Class Initialization	17
3.12 NotAvailable Exception	18
3.13 write Runnable	19
3.14 Test Case Coverage Justification	19
4. Test Implementation and Results	19
4.1 Test Environment Setup	19
4.2 Implementation Details and Code Snippets	20
4.3 Test Execution Results	21
4.4 Analysis of Results	27
5. Limitations of the Testing Work	28
6. Conclusion	29

1. Project Description

1.1 System Overview

This testing assignment will use a Java console-based hotel management system as the project. The system is supposed to take care of hotel reservations, food orders, billing, and checking out. It has a text-based menu that lets users interact with seven main features: ordering food, checking out, canceling a reservation, booking a hotel, seeing room availability, seeing room details, and leaving the system.

There are four types of rooms at the hotel, and they all have fifty rooms. Luxury double rooms (rooms 101 to 110) cost 4000 TK per day for two people. Deluxe Double rooms (rooms 111 to 130) cost 3000 TK per day for two people. Luxury single rooms (rooms 131 to 140) cost 2200 TK per day for one person. One person can stay in a Deluxe Single room (rooms 141 to 150) for 1200 TK a day. Guests can order food from an in-house menu while they are there. The final bill at checkout is the sum of the room price and any food costs that have been added.

The system uses Java object serialization to store its data. ObjectOutputStream writes all of the booking information to a binary file called hotel.dat and loaded again when the program starts. To make sure that the data in the system is always correct, guest inputs for names, phone numbers, and gender are checked before they are saved.

1.2 Project Structure — Classes and Methods

The project is made up of eleven source files, which are all in the "project" package. The table below shows what each class does, what its methods are, and what each method's purpose is.

Class	Method / Field	Description
Bill	calculate(int type, int num, holder h)	Computes and returns the total bill for a given room as a float by adding the fixed room charge for the type to the food bill accumulated in the room object.
Doubleroom	Doubleroom(n1,c1,g1,n2,c2,g2)	Constructor that initializes all six guest fields for two guests and creates an empty food list.
Doubleroom	getFoodBill()	Iterates the ArrayList<Food> and returns the sum of all food item prices as a float.
FileSystem	save(holder h)	Serializes the holder object to hotel.dat using ObjectOutputStream.
FileSystem	load()	Deserializes and returns the holder object from hotel.dat; returns a new holder if file does not exist.

Class	Method / Field	Description
Food	Food()	Constructor that displays the food menu, reads item number and quantity from Scanner, and calculates the price using a switch statement.
Food	getPrice()	Returns the price of the food item as a float.
Hotel	getIndex(int type, int roomNo)	Converts a guest-entered room number to a zero-based array index; returns -1 for invalid inputs.
Hotel	countAvailable(Object[] rooms)	Iterates a room array and counts the number of null (unbooked) slots.
Hotel	displayRoomDetails()	Reads room type from Scanner and prints detailed information about that room category.
Hotel	displayAvailability()	Prints the number of available rooms in each of the four categories.
Hotel	bookRoom()	Reads room type, room number, and guest details from Scanner and creates the appropriate room object.
Hotel	orderFood()	Reads room details from Scanner, verifies the room is booked, and adds a new Food object to the room's list.
Hotel	checkout()	Reads room details, calls Bill.calculate(), prints the total, and saves the data.
Hotel	cancelBooking()	Reads room details and sets the corresponding array slot to null.
NotAvailable	toString()	Overrides Exception.toString() and returns the string 'Room Not Available'.
Project	main(String[] args)	Entry point; runs a loop displaying the main menu and dispatching to Hotel methods based on user choice.
Singleroom	Singleroom(name, contact, gender)	Constructor that initializes the three guest fields and creates an empty food list.
Singleroom	getFoodBill()	Iterates the ArrayList<Food> and returns the sum of all food item prices as a float.
holder	(fields)	Serializable data container with four arrays: luxuryDouble[10], deluxeDouble[20], luxurySingle[10], deluxeSingle[10].
write	run()	Implements Runnable.run(); prints 'Backup saved.' to stdout.

2. Identification of the Testing Scope

2.1 Scope of Testing

Due to design constraints, not all components of this project can undergo automated unit testing. The main problem is that a lot of people use Scanner (java.util.Scanner) to read user input in constructors and method bodies. Methods like Hotel.bookRoom(), Hotel.orderFood(), Hotel.checkout(), and the Food() constructor read System.In a headless JUnit test environment, they can't be called directly unless you mock or change the standard input.

Also, FileSystem.save() and FileSystem.load() write files to and read files from disk. This is part of integration testing, not unit testing. Project.main() is just an event loop, so there are no return values to check.

The three static methods of the Validator class, the constructors and getFoodBill() methods of Singleroom and Doubleroom, the static calculate() method of the Bill class, the static getIndex() and countAvailable() methods of the Hotel class, the initialization behavior of the holder class, the custom exception NotAvailable, and the Runnable implementation in the write class are all included in the project's unit testing scope. All of the project's significant business logic is contained in these components, which are completely tested and independent of Scanner or file I/O.

2.2 Testing Perspectives per Method

Validator.isValidName()

This technique validates visitor names using the regular expression `[a-zA-Z ]+`. Among the testing viewpoints are: (1) To ensure that all acceptable name forms are allowed, valid inputs are names made up only of letters and spaces in different casing combinations. (2) In order to verify that the regex appropriately rejects non-alphabetic characters, invalid inputs include names that contain numerals, special characters, hyphens, and dots. (3) Because the `+` quantifier requires at least one character, edge cases like the empty string are crucial; supplying an empty string should result in false. Because accepting an invalid name might destroy the guest record kept in the data file, it is crucial to thoroughly test this procedure.

Validator.isValidPhone()

This technique verifies Bangladeshi mobile numbers using the regex `01[3-9][0-9]{8}`. Among the testing viewpoints are: (1) All valid prefixes (013 through 019) must be examined because the regex demands that the third digit fall between 3 and 9. (2) Invalid prefixes: Since the third digits of the prefixes 010, 011, and 012 fall outside of `[3-9]`, they must be discarded. (3)

Boundary length: precisely 11 digits are needed for the regex. One-below and one-above boundary values are represented by inputs of 10 and 12 digits, which must both be rejected. (4) Format violations: an empty string and non-numeric characters must be rejected. This approach is essential since the hotel would not be able to get in touch with a visitor if their phone number was invalid.

Validator.isValidGender()

This function determines whether the input is "male" or "female" using `String.equalsIgnoreCase()`. Among the testing viewpoints are: (1) All common casing variations (title case, lowercase, and uppercase) must be permitted for case-insensitive valid inputs. (2) Mixed case: in order to confirm the case-insensitive behavior, odd casing like "mALe" must also be tolerated. (3) Invalid values: single-letter abbreviations, empty strings, and strings like "Other" must all be rejected. This guarantees that the system only incorporates standardized gender values.

Singleroom Constructor

Three parameters (name, contact, and gender) are passed to package-level fields by the constructor. Among the testing viewpoints are: (1) Field storage: To guarantee that each field is placed in the appropriate variable, each field must be checked separately. (2) Initialization of the food list: if the food `ArrayList` is not initialized during construction, a `NullPointerException` will occur when food is ordered later. (3) Object independence: two distinct In order to ensure that static fields are not inadvertently used, Singleroom objects must not exchange field values.

Singleroom.getFoodBill() and Doubleroom.getFoodBill()

Both approaches total the prices while iterating through an `ArrayList`. Among the testing viewpoints are: (1) Empty list: When no food has been ordered, the procedure should return precisely 0.0; this is a crucial edge case since a checkout shouldn't add phantom charges. (2) Single item: To ensure that the loop runs correctly with a single element, the method should return the precise price of a single item. (3) several items: To test the accumulation logic, the total of several items must be accurate. (4) Floating-point precision: decimal inputs must add up accurately without floating-point rounding mistakes leading to inaccurate invoices.

Doubleroom Constructor

Like the Singleroom constructor, but with six fields rather of three, same viewpoints are applicable. To make sure no parameters are overwritten or switched during assignment, each guest's name, contact, and gender fields must be tested separately.

Bill.calculate()

Using an if-else chain on the type parameter, this technique embeds four preset room charges as constants (4000, 3000, 2200, and 1200). Among the testing viewpoints are: (1) Per-type correctness: To ensure that the right charge constant is chosen, each of the four room types needs to be tested. (2) Food charge inclusion: To ensure that the food bill is appropriately added to the room charge, each kind must also be tested with food items. (3) Price hierarchy: the four fees must continue to be in the anticipated declining order. (4) Index independence: Since the fee is based only on type, the bill must be the same regardless of which array index (i.e., room number) is utilized. This method's mistakes would immediately result in overcharged or undercharged guests.

Hotel.getIndex()

This method uses math to map a room number to an array index ($\text{index} = \text{roomNo} - \text{base}$). Boundary value analysis dominates the testing perspectives: (1) Lower boundary: index 0 must be returned by the first room in each range that is valid. (2) Upper boundary: the maximum valid index must be returned by the final valid room in each range. (3) Above-upper boundary: the room number one above the valid range must return -1; (4) Below-lower boundary: the room number one below the valid range must return -1; (5) Invalid type: a type value outside of 1-4 must return -1; (6) Negative input: a negative room number must return -1. A defect in this method, which serves as the entry point for all room activities, could silently corrupt data by reading or writing to the incorrect array index.

Hotel.countAvailable()

An Object array's null items are counted using this technique. Among the testing viewpoints are: (1) All-null: the count should match the array length when every room is available. (2) All-non-null: the count should be 0 when every room is reserved. (3) Mixed: the right partial count should be returned by a partially booked array. (4) A single-element array in both states is the edge case. (5) Integration with holder: all of the newly created holder arrays should begin completely available, indicating that Java's default behavior has initialized the arrays to null.

holder Class

The central data store is the holder class. Its array widths are more than simply a detail; they actually dictate the range of array indices that `getIndex()` can employ. Testing viewpoints consist of: (1) Accurate dimensions: every array needs to have the exact length that corresponds to the room range. (2) Null initialization: all slots must be initialized to null by default, ready to take reservations, and all four arrays must be non-null. (3) Mutability: slots must support setting back to null for cancellations and accept room assignments.

NotAvailable and write

ToString() is overridden by a special exception class called NotAvailable. Since this is the message that would be shown to the user, the exact string given by toString() is its primary testing perspective. The write class is an implementation of Runnable, and its concurrency contract is validated by verifying whether it can run in a thread error-free.

3. Test Case Design

The testing viewpoints listed in Section 2 served as the direct basis for the design of the test cases. Cases covering boundary conditions, invalid or edge-case inputs, and typical expected behavior were developed for each approach. All of the test cases are arranged by method in the tables below. The test case identifier, the test's objective, the particular input values utilized, the anticipated result, and the testing scenario or condition being checked are all included in each table.

3.1 isValidName(): Validator

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 01	Verify a valid single-word name is accepted	'Alice'	true	Valid input, letters only
TC 02	Verify a full name with one space is accepted	'Alice Rahman'	true	Valid input, letters with space
TC 03	Verify a name with multiple spaces is accepted	'Md Al Amin'	true	Valid input, multiple spaces allowed
TC 04	Verify all-lowercase name is accepted	'john doe'	true	Valid input, lowercase letters
TC 05	Verify all-uppercase name is accepted	'JOHN DOE'	true	Valid input, uppercase letters
TC 06	Verify name containing digits is rejected	'Alice123'	false	Invalid input, digits not in pattern

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 07	Verify name with special character @ is rejected	'Alice@Khan'	false	Invalid input, special character
TC 08	Verify name with hyphen is rejected	'Mary-Jane'	false	Invalid input, hyphen not in pattern
TC 09	Verify empty string is rejected	" (empty)	false	Edge case, empty string, + requires >= 1 char
TC 10	Verify name with a dot is accepted	'Dr. Smith'	true	Valid input

3.2 isValidPhone(): Validator

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 11	Verify phone with 017 prefix is accepted	'01711223344'	true	Valid BD prefix, 7 is in [3-9]
TC 12	Verify phone with 018 prefix is accepted	'01812345678'	true	Valid BD prefix, 8 is in [3-9]
TC 13	Verify phone with 019 prefix is accepted	'01912345678'	true	Valid BD prefix, 9 is in [3-9]
TC 14	Verify phone with 013 prefix is accepted	'01312345678'	true	Boundary, 3 is the minimum in [3-9]
TC 15	Verify phone with 016 prefix is accepted	'01612345678'	true	Valid BD prefix, 6 is in [3-9]
TC 16	Verify 010 prefix is rejected	'01012345678'	false	Invalid prefix, 0 is not in [3-9]
TC 17	Verify 011 prefix is rejected	'01112345678'	false	Invalid prefix, 1 is not in [3-9]
TC 18	Verify 012 prefix is rejected	'01212345678'	false	Invalid prefix, 2 is not in [3-9]

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 19	Verify 10-digit phone (too short) is rejected	'0171122334'	false	Boundary, one digit short of 11
TC 20	Verify 12-digit phone (too long) is rejected	'017112233445'	false	Boundary, one digit over 11
TC 21	Verify phone containing letters is rejected	'017ABCDEFGH'	false	Invalid input, alphabetic characters
TC 22	Verify empty string is rejected	" (empty)	false	Edge case, empty string fails regex
TC 23	Verify phone not starting with 01 is rejected	'02712345678'	false	Invalid prefix, does not start with 01

3.3 isValidGender(): Validator

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 24	Verify 'Male' (title case) is accepted	'Male'	true	Valid input, matches equalsIgnoreCase
TC 25	Verify 'Female' (title case) is accepted	'Female'	true	Valid input, matches equalsIgnoreCase
TC 26	Verify 'male' (lowercase) is accepted	'male'	true	Case-insensitive, lowercase match
TC 27	Verify 'female' (lowercase) is accepted	'female'	true	Case-insensitive, lowercase match
TC 28	Verify 'MALE' (uppercase) is accepted	'MALE'	true	Case-insensitive, uppercase match
TC 29	Verify 'FEMALE' (uppercase) is accepted	'FEMALE'	true	Case-insensitive, uppercase match
TC 30	Verify 'Other' is rejected	'Other'	false	Invalid, not 'male' or 'female'

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 31	Verify 'M' abbreviation is rejected	'M'	false	Invalid, abbreviation not accepted
TC 32	Verify 'F' abbreviation is rejected	'F'	false	Invalid, abbreviation not accepted
TC 33	Verify empty string is rejected	" (empty)	false	Edge case, empty string
TC 34	Verify 'mALe' mixed case is accepted	'mALe'	true	Case-insensitive, unusual mixed casing
TC 35	Verify 'fEmAlE' mixed case is accepted	'fEmAlE'	true	Case-insensitive, unusual mixed casing

3.4 Singleroom Constructor

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 01	Verify name field is stored correctly by constructor	name='Alice', contact, gender	s.name equals 'Alice'	Field storage, constructor assignment
TC 02	Verify contact field is stored correctly	name, contact='01711223344', gender	s.contact equals '01711223344'	Field storage, constructor assignment
TC 03	Verify gender field is stored correctly	name, contact, gender='Female'	s.gender equals 'Female'	Field storage, constructor assignment
TC 04	Verify food list is initialized as empty	New Singleroom object created	s.food is not null, size is 0	Initialization, ArrayList must start empty
TC 05	Verify two objects have independent state	s1 and s2 with different data	All fields of s1 differ from s2	Object independence, no shared references

3.5 Singleroom.getFoodBill()

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 06	Verify bill is 0.0 when no food has been ordered	No Food objects in list	Returns 0.0	Edge case, empty food list
TC 07	Verify bill equals price of single food item	One Food item: price = 40.0 TK	Returns 40.0	Single item, sum of one element
TC 08	Verify bill is correct sum for three food items	Tea(20)+Sandwich(50)+Noodles(70)	Returns 140.0	Multiple items, sum of all prices
TC 09	Verify running total increases correctly after each add	Add Coke(30), then add Pasta(60)	30.0 after first, 90.0 after second	State change, bill updates on each addition
TC 10	Verify bill is correct for large number of items	5 Food items each priced 70.0 TK	Returns 350.0	Volume test, many items summed correctly

3.6 Doubleroom Constructor

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 11	Verify guest1 name field is stored correctly	name1='Alice', 5 other params	d.name1 equals 'Alice'	Field storage, first guest name
TC 12	Verify guest1 contact field is stored correctly	contact1='01711111111'	d.contact1 equals '01711111111'	Field storage, first guest contact
TC 13	Verify guest1 gender field is stored correctly	gender1='Female'	d.gender1 equals 'Female'	Field storage, first guest gender
TC 14	Verify guest2 name field is stored correctly	name2='Bob'	d.name2 equals 'Bob'	Field storage, second guest name
TC 15	Verify guest2 contact field is stored correctly	contact2='01722222222'	d.contact2 equals '01722222222'	Field storage, second guest contact

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 16	Verify guest2 gender field is stored correctly	gender2='Male'	d.gender2 equals 'Male'	Field storage, second guest gender
TC 17	Verify food list is initialized as empty	New Doubleroom object created	d.food is not null, size is 0	Initialization, ArrayList must start empty
TC 18	Verify two Doubleroom	d1 and d2 with different guest data	All fields of d1 differ from d2	Object independence, no shared references

3.7 Doubleroom.getFoodBill()

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 19	Verify bill is 0.0 when no food ordered	No Food objects in list	Returns 0.0	Edge case, empty food list
TC 20	Verify bill equals price of one food item	One Food item: price = 50.0 TK	Returns 50.0	Single item, sum of one element
TC 21	Verify bill is correct sum for three items	Tea(20)+Coffee(40)+Pasta(60)	Returns 120.0	Multiple items, all prices summed
TC 22	Verify decimal food prices are summed correctly	Item1=25.5 TK, Item2=14.5 TK	Returns 40.0	Floating-point precision, decimal addition

3.8 Bill.calculate()

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 01	Verify type 1 room charge is 4000 when no food ordered	type=1, num=0, no food items	Returns 4000.0	Luxury Double, fixed room charge only
TC 02	Verify type 1 total includes food charge	type=1, Coffee(40)+Noodles(70)	Returns 4110.0	Luxury Double, room + food sum
TC 03	Verify type 1 bill is correct at a different array index	type=1, num=5, no food	Returns 4000.0	Index independence, charge same at any index

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 04	Verify type 2 room charge is 3000 when no food ordered	type=2, num=0, no food items	Returns 3000.0	Deluxe Double, fixed room charge only
TC 05	Verify type 2 total includes food charge	type=2, Tea(20)+Sandwich(50)	Returns 3070.0	Deluxe Double, room + food sum
TC 06	Verify type 2 charge is less than type 1	Compare type=1 and type=2 results	type2 result < type1 result	Price hierarchy, Deluxe < Luxury
TC 07	Verify type 3 room charge is 2200 when no food ordered	type=3, num=0, no food items	Returns 2200.0	Luxury Single, fixed room charge only
TC 08	Verify type 3 total includes food charge	type=3, Pasta(60)+Coke(30)	Returns 2290.0	Luxury Single, room + food sum
TC 09	Verify type 3 bill is correct at a different array index	type=3, num=3, no food	Returns 2200.0	Index independence, charge same at any index
TC 10	Verify type 4 room charge is 1200 when no food ordered	type=4, num=0, no food items	Returns 1200.0	Deluxe Single, fixed room charge only
TC 11	Verify type 4 total includes food charge	type=4, Tea(20)+Coffee(40)+Noodles(70)	Returns 1330.0	Deluxe Single, room + food sum
TC 12	Verify full price hierarchy across all 4 types	All 4 types compared	t4 < t3 < t2 < t1	Ordering, all charges strictly decreasing
TC 13	Verify exact charge values for all 4 types	All 4 types, no food	4000, 3000, 2200, 1200	Correctness, all charge constants verified
TC 14	Verify total equals room charge plus getFoodBill()	type=1, one food item: 100.0 TK	Returns 4100.0	Integration, bill = roomCharge + foodBill
TC 15	Verify multiple same food items sum correctly	type=4, three Tea items (3x20 TK)	Returns 1260.0	Multiple items, repeated orders summed

3.9 Hotel.getIndex()

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 01	Verify first room of type 1 returns index 0	type=1, roomNo=101	Returns 0	Boundary, lower bound of type 1 range
TC 02	Verify middle room of type 1 returns correct index	type=1, roomNo=105	Returns 4	Valid range, index = 105 - 101
TC 03	Verify last room of type 1 returns index 9	type=1, roomNo=110	Returns 9	Boundary, upper bound of type 1 range
TC 04	Verify room below type 1 range returns -1	type=1, roomNo=100	Returns -1	Boundary, one below lower bound
TC 05	Verify room above type 1 range returns -1	type=1, roomNo=111	Returns -1	Boundary, one above upper bound
TC 06	Verify first room of type 2 returns index 0	type=2, roomNo=111	Returns 0	Boundary, lower bound of type 2 range
TC 07	Verify middle room of type 2 returns correct index	type=2, roomNo=120	Returns 9	Valid range, index = 120 - 111
TC 08	Verify last room of type 2 returns index 19	type=2, roomNo=130	Returns 19	Boundary, upper bound of type 2 range
TC 09	Verify room below type 2 range returns -1	type=2, roomNo=110	Returns -1	Boundary, one below lower bound
TC 10	Verify room above type 2 range returns -1	type=2, roomNo=131	Returns -1	Boundary, one above upper bound
TC 11	Verify first room of type 3 returns index 0	type=3, roomNo=131	Returns 0	Boundary, lower bound of type 3 range
TC 12	Verify middle room of type 3 returns correct index	type=3, roomNo=135	Returns 4	Valid range, index = 135 - 131

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 13	Verify last room of type 3 returns index 9	type=3, roomNo=140	Returns 9	Boundary, upper bound of type 3 range
TC 14	Verify room below type 3 range returns -1	type=3, roomNo=130	Returns -1	Boundary, one below lower bound
TC 15	Verify room above type 3 range returns -1	type=3, roomNo=141	Returns -1	Boundary, one above upper bound
TC 16	Verify first room of type 4 returns index 0	type=4, roomNo=141	Returns 0	Boundary, lower bound of type 4 range
TC 17	Verify middle room of type 4 returns correct index	type=4, roomNo=145	Returns 4	Valid range, index = 145 - 141
TC 18	Verify last room of type 4 returns index 9	type=4, roomNo=150	Returns 9	Boundary, upper bound of type 4 range
TC 19	Verify room below type 4 range returns -1	type=4, roomNo=140	Returns -1	Boundary, one below lower bound
TC 20	Verify room above type 4 range returns -1	type=4, roomNo=151	Returns -1	Boundary, one above upper bound
TC 21	Verify invalid room type 5 returns -1	type=5, roomNo=101	Returns -1	Invalid input, type 5 does not exist
TC 22	Verify invalid room type 0 returns -1	type=0, roomNo=101	Returns -1	Invalid input, type 0 does not exist
TC 23	Verify negative room number returns -1	type=1, roomNo=-1	Returns -1	Edge case, negative room number

3.10 Hotel.countAvailable()

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 24	Verify all-null array returns full array length	10-element array, all null	Returns 10	All rooms free, every slot is null

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 25	Verify fully booked array returns 0	5-element array, all non-null	Returns 0	No rooms free, no null slots
TC 26	Verify mixed array counts free slots correctly	5 rooms, 3 booked (index 0,2,4)	Returns 2	Mixed state, counts only null slots
TC 27	Verify single-element null array returns 1	1-element array, null	Returns 1	Edge case, minimum array, free
TC 28	Verify single-element booked array returns 0	1-element array, non-null	Returns 0	Edge case, minimum array, booked
TC 29	Verify luxuryDouble array has 10 free rooms initially	new holder().luxuryDouble	Returns 10	Initial state, all 10 rooms free
TC 30	Verify deluxeDouble array has 20 free rooms initially	new holder().deluxeDouble	Returns 20	Initial state, all 20 rooms free
TC 31	Verify luxurySingle array has 10 free rooms initially	new holder().luxurySingle	Returns 10	Initial state, all 10 rooms free
TC 32	Verify deluxeSingle array has 10 free rooms initially	new holder().deluxeSingle	Returns 10	Initial state, all 10 rooms free

3.11 holder Class Initialization

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 33	Verify luxuryDouble array has exactly 10 slots	new holder()	luxuryDouble.length equals 10	Array sizing, matches room range 101-110
TC 34	Verify deluxeDouble array has exactly 20 slots	new holder()	deluxeDouble.length equals 20	Array sizing, matches room range 111-130
TC 35	Verify luxurySingle array has exactly 10 slots	new holder()	luxurySingle.length equals 10	Array sizing, matches room range 131-140

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 36	Verify deluxeSingle array has exactly 10 slots	new holder()	deluxeSingle.length equals 10	Array sizing, matches room range 141-150
TC 37	Verify all four arrays are initialized (not null)	new holder()	All four array references not null	Initialization, arrays created in constructor
TC 38	Verify a Doubleroom can be assigned to luxuryDouble	Assign Doubleroom to index 0	luxuryDouble[0] is not null	State assignment, booking a room
TC 39	Verify a Singleroom can be assigned to luxurySingle	Assign Singleroom to index 0	luxurySingle[0] is not null	State assignment, booking a room
TC 40	Verify setting a slot to null cancels booking	Assign then set to null	Slot becomes null	State mutation, cancelling a booking

3.12 NotAvailable Exception

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 41	Verify NotAvailable is a subclass of Exception	new NotAvailable()	instanceof Exception is true	Inheritance, extends Exception correctly
TC 42	Verify toString() returns exact message	new NotAvailable().toString()	'Room Not Available'	Return value, overridden toString()
TC 43	Verify the exception can be thrown and caught	assertThrows block	NotAvailable is caught	Throwable, valid checked exception
TC 44	Verify toString() result is not null or empty	new NotAvailable().toString()	Not null, length greater than 0	Robustness, non-empty error message

3.13 Write Runnable

TC ID	Purpose	Input Values	Expected Output	Testing Scenario
TC 45	Verify write implements the Runnable interface	new write()	instanceof Runnable is true	Interface, Runnable contract satisfied
TC 46	Verify run() executes without throwing any exception	w.run()	No exception thrown	Normal execution, run() is safe to call
TC 47	Verify write can execute inside a Thread	new Thread(w).start(); t.join()	Thread completes normally	Concurrency, Runnable works in a Thread

3.14 Test Case Coverage Justification

Six different testing viewpoints are covered by the collection of test cases created above. First, TC01–TC05 (name), TC11–TC15 (phone), TC24–TC29 (gender), TC01–TC05 (Singleroom), TC11–TC18 (Doubleroom), and TC01–TC15 (Bill) all accomplish valid input coverage by verifying that proper inputs result in right outputs. Second, TC06–TC10 (name), TC16–TC23 (phone), and TC30–TC33 (gender) accomplish incorrect input coverage, guaranteeing that the system rejects erroneous data. Third, Hotel.getIndex() is the function that uses boundary value analysis the most extensively. For each of the four types of rooms, tests are created at the exact lower bound, a medium value, the exact higher bound, one below the lower bound, one above the upper bound, and a full five-point boundary sweep. Fourth, TC09 (empty name), TC22 (empty phone), TC33 (empty gender), TC06 and TC19 (zero food bill), and TC27–TC28 (single-element array) all target inputs that indicate severe or degenerate circumstances. Fifth, state verification is handled by TC40, which verifies that setting a slot to null appropriately cancels a booking, and the constructor tests for Singleroom and Doubleroom, which validate that each field is independently saved. Sixth, TC14 (Bill.calculate() integrating with getFoodBill()) and TC29–TC32 (countAvailable() applied to real holder arrays) address integration behavior and verify that the classes work as intended.

4. Test Implementation and Results

4.1 Test Environment Setup

JUnit 5 (JUnit Jupiter) was used in the Eclipse IDE to implement the tests. The test source files were arranged in the package "project" to correspond with the package of the source classes being tested, and they were placed in a different source folder called "test" under the same project. Using Eclipse's 'Add Libraries' option, JUnit 5 was chosen from the library list and

added to the project's build path. The entire test suite was run by right-clicking the project in the Eclipse Package Explorer and choosing 'Run As > JUnit Test'. Four test files were created: ValidatorTest.java, RoomTest.java, BillTest.java, and HotelUtilityTest.java. 119 tests were ran with 0 errors and 0 failures, according to the JUnit results view.

4.2 Implementation Details and Code Snippets

Standard JUnit 5 Test Structure

Every test function utilizes a static assertion method from `org.junit.jupiter.api.Assertions` and is annotated with `@Test`. A sample set of tests from `ValidatorTest.java` is displayed in the following snippet:

```
1 package project;
2
3 import org.junit.jupiter.api.*;
4
5
6 @DisplayName("Validator Class Tests")
7 public class ValidatorTest {
8
9     // 1. isValidName() Tests
10
11     @Test
12     @DisplayName("TC01 - Valid single word name returns true")
13     void testValidNameSingleWord() {
14         assertTrue(Validator.isValidName("Alice"));
15     }
16
17     @Test
18     @DisplayName("TC02 - Valid full name with space returns true")
19     void testValidNameWithSpace() {
20         assertTrue(Validator.isValidName("Alice Rahman"));
21     }
22
23     @Test
24     @DisplayName("TC02 - Singleroom constructor stores contact correctly")
25     void testSingleroomStoreContact() {
26         Singleroom s = new Singleroom("Alice", "01711223344", "Female");
27         assertEquals("01711223344", s.contact);
28     }
29
30     @Test
31     @DisplayName("TC03 - Singleroom constructor stores gender correctly")
32     void testSingleroomStoreGender() {
33         Singleroom s = new Singleroom("Alice", "01711223344", "Female");
34         assertEquals("Female", s.gender);
35     }
36
37     @Test
38     @DisplayName("TC04 - Singleroom food list is initialized and empty on creation")
39     void testSingleroomFoodListInitiallyEmpty() {
40         Singleroom s = new Singleroom("Bob", "01812345678", "Male");
41         assertNotNull(s.food);
42         assertEquals(0, s.food.size());
43     }
44
45     @Test
46     @DisplayName("TC11 - Valid 017 prefix phone returns true")
47     void testValidPhone017() {
48         assertTrue(Validator.isValidPhone("01711223344"));
49     }
50
51     @Test
52     @DisplayName("TC12 - Valid 018 prefix phone returns true")
53     void testValidPhone018() {
54         assertTrue(Validator.isValidPhone("01812345678"));
55     }
56
57     @Test
58     @DisplayName("TC13 - Valid 019 prefix phone returns true")
59     void testValidPhone019() {
60         assertTrue(Validator.isValidPhone("01912345678"));
61     }
62
63     @Test
64     @DisplayName("TC05 - Type 2 (Deluxe Double): bill = 3000 + food charge")
65     void testDeluxeDoubleWithFood() throws Exception {
66         h.deluxeDouble[0].food.add(makeFoodWithPrice(20.0f)); // Tea
67         h.deluxeDouble[0].food.add(makeFoodWithPrice(50.0f)); // Sandwich
68         // Room: 3000 + Food: 70 = 3070
69         float bill = Bill.calculate(2, 0, h);
70         assertEquals(3070.0, bill, 0.001);
71     }
72 }
```

Reflection Technique for Food Object Creation

Direct instantiation in a unit test is not possible since the `Food()` constructor uses `Scanner` to read from `System.in`. Java Reflection and `sun.misc.Unsafe.allocateInstance()` were used to create a helper function that tested `getFoodBill()` on `Singleroom` and `Doubleroom`. This method sets the price field directly through reflection after allocating a `Food` object in memory without running its constructor. This excerpt is taken from `RoomTest.java`:

```
// — Helper: create Food with known price via Reflection —
private Food makeFoodWithPrice(float price) throws Exception {
    Food f = (Food) sun.misc.Unsafe.class
        .getDeclaredMethod("allocateInstance", Class.class)
        .invoke(getUnsafe(), Food.class);
    Field priceField = Food.class.getDeclaredField("price");
    priceField.setAccessible(true);
    priceField.set(f, price);
    return f;
}

private sun.misc.Unsafe getUnsafe() throws Exception {
    Field f = sun.misc.Unsafe.class.getDeclaredField("theUnsafe");
    f.setAccessible(true);
    return (sun.misc.Unsafe) f.get(null);
}
```

Test Setup Using @BeforeEach

For BillTest.java, a @BeforeEach method was used to pre-configure a holder object with one booked room of each type before each test, eliminating repetitive setup code and ensuring a clean state per test:

```
@BeforeEach
void setUp() {
    h = new holder();

    // Pre-book one room of each type at index 0
    h.luxuryDouble[0] = new Doubleroom("Alice", "0171111111", "Female",
                                       "Bob", "0172222222", "Male");

    h.deluxeDouble[0] = new Doubleroom("Carol", "0173333333", "Female",
                                       "Dave", "0174444444", "Male");

    h.luxurySingle[0] = new Singleroom("Eve", "0175555555", "Female");
    h.deluxeSingle[0] = new Singleroom("Frank", "0176666666", "Male");
}
```

4.3 Test Execution Results

The following tables present the results of executing each test suite. Since all 119 tests passed, the actual output matches the expected output in every case.

ValidatorTest: isValidName() Results (TC01–TC10)

TC ID	Expected Output	Actual Output	Result
TC01	true	true	PASS
TC02	true	true	PASS
TC03	true	true	PASS
TC04	true	true	PASS
TC05	true	true	PASS

TC ID	Expected Output	Actual Output	Result
TC06	false	false	PASS
TC07	false	false	PASS
TC08	false	false	PASS
TC09	false	false	PASS
TC10	true	true	PASS

ValidatorTest: isValidPhone() Results (TC11–TC23)

TC ID	Expected Output	Actual Output	Result
TC11	true	true	PASS
TC12	true	true	PASS
TC13	true	true	PASS
TC14	true	true	PASS
TC15	true	true	PASS
TC16	false	false	PASS
TC17	false	false	PASS
TC18	false	false	PASS
TC19	false	false	PASS
TC20	false	false	PASS
TC21	false	false	PASS
TC22	false	false	PASS
TC23	false	false	PASS

ValidatorTest: isValidGender() Results (TC24–TC35)

TC ID	Expected Output	Actual Output	Result
TC24	true	true	PASS
TC25	true	true	PASS
TC26	true	true	PASS
TC27	true	true	PASS
TC28	true	true	PASS
TC29	true	true	PASS
TC30	false	false	PASS
TC31	false	false	PASS
TC32	false	false	PASS
TC33	false	false	PASS
TC34	true	true	PASS
TC35	true	true	PASS

RoomTest: Singleroom Results (TC01–TC10)

TC ID	Expected Output	Actual Output	Result
TC01	s.name = 'Alice'	s.name = 'Alice'	PASS
TC02	s.contact = '01711223344'	s.contact = '01711223344'	PASS
TC03	s.gender = 'Female'	s.gender = 'Female'	PASS
TC04	s.food not null, size = 0	s.food not null, size = 0	PASS
TC05	All fields of s1 differ from s2	All fields of s1 differ from s2	PASS
TC06	getFoodBill() = 0.0	0.0	PASS
TC07	getFoodBill() = 40.0	40.0	PASS
TC08	getFoodBill() = 140.0	140.0	PASS
TC09	30.0 then 90.0	30.0 then 90.0	PASS
TC10	getFoodBill() = 350.0	350.0	PASS

RoomTest: Doubleroom Results (TC11–TC22)

TC ID	Expected Output	Actual Output	Result
TC11	d.name1 = 'Alice'	d.name1 = 'Alice'	PASS
TC12	d.contact1 = '01711111111'	d.contact1 = '01711111111'	PASS
TC13	d.gender1 = 'Female'	d.gender1 = 'Female'	PASS
TC14	d.name2 = 'Bob'	d.name2 = 'Bob'	PASS
TC15	d.contact2 = '01722222222'	d.contact2 = '01722222222'	PASS
TC16	d.gender2 = 'Male'	d.gender2 = 'Male'	PASS
TC17	d.food not null, size = 0	d.food not null, size = 0	PASS
TC18	All fields of d1 differ from d2	All fields of d1 differ from d2	PASS
TC19	getFoodBill() = 0.0	0.0	PASS
TC20	getFoodBill() = 50.0	50.0	PASS
TC21	getFoodBill() = 120.0	120.0	PASS
TC22	getFoodBill() = 40.0	40.0	PASS

BillTest Results (TC01–TC15)

TC ID	Expected Output	Actual Output	Result
TC01	4000.0	4000.0	PASS
TC02	4110.0	4110.0	PASS
TC03	4000.0	4000.0	PASS
TC04	3000.0	3000.0	PASS
TC05	3070.0	3070.0	PASS
TC06	type2 < type1	3000 < 4000	PASS
TC07	2200.0	2200.0	PASS
TC08	2290.0	2290.0	PASS
TC09	2200.0	2200.0	PASS
TC10	1200.0	1200.0	PASS
TC11	1330.0	1330.0	PASS

TC ID	Expected Output	Actual Output	Result
TC12	t4 < t3 < t2 < t1	1200<2200<3000<4000	PASS
TC13	4000/3000/2200/1200	4000/3000/2200/1200	PASS
TC14	4100.0	4100.0	PASS
TC15	1260.0	1260.0	PASS

HotelUtilityTest: getIndex() Results (TC01–TC23)

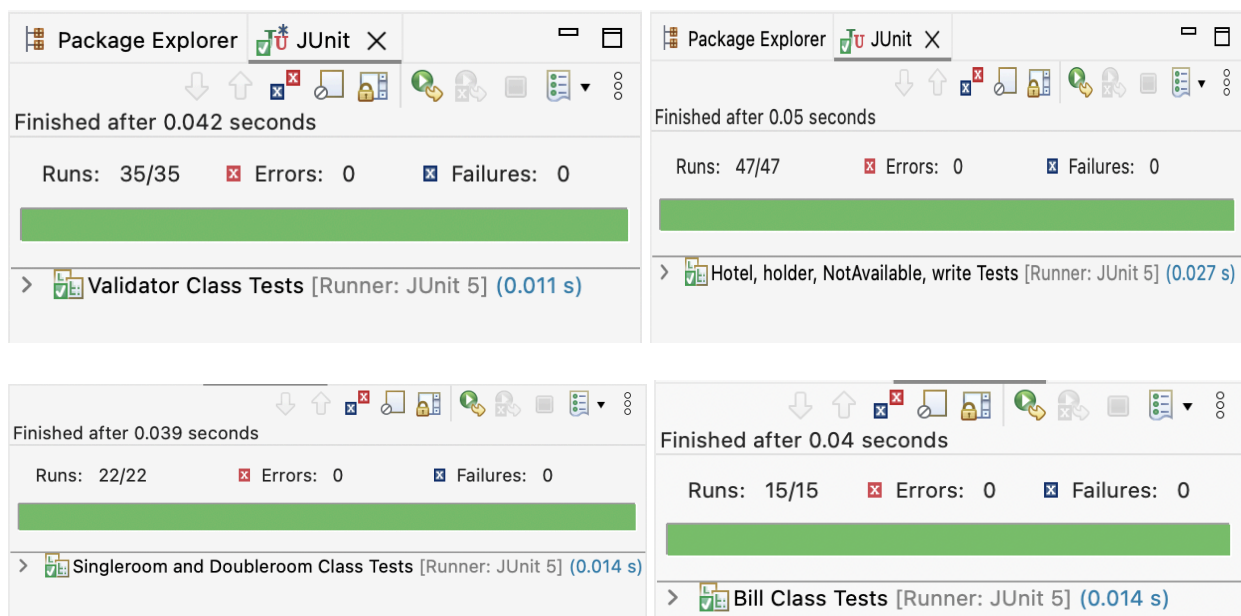
TC ID	Expected Output	Actual Output	Result
TC01	0	0	PASS
TC02	4	4	PASS
TC03	9	9	PASS
TC04	-1	-1	PASS
TC05	-1	-1	PASS
TC06	0	0	PASS
TC07	9	9	PASS
TC08	19	19	PASS
TC09	-1	-1	PASS
TC10	-1	-1	PASS
TC11	0	0	PASS
TC12	4	4	PASS
TC13	9	9	PASS
TC14	-1	-1	PASS
TC15	-1	-1	PASS
TC16	0	0	PASS
TC17	4	4	PASS
TC18	9	9	PASS
TC19	-1	-1	PASS

TC ID	Expected Output	Actual Output	Result
TC20	-1	-1	PASS
TC21	-1	-1	PASS
TC22	-1	-1	PASS
TC23	-1	-1	PASS

HotelUtilityTest: countAvailable(), holder, NotAvailable, write Results (TC24–TC47)

TC ID	Expected Output	Actual Output	Result
TC24	10	10	PASS
TC25	0	0	PASS
TC26	2	2	PASS
TC27	1	1	PASS
TC28	0	0	PASS
TC29	10	10	PASS
TC30	20	20	PASS
TC31	10	10	PASS
TC32	10	10	PASS
TC33	length = 10	length = 10	PASS
TC34	length = 20	length = 20	PASS
TC35	length = 10	length = 10	PASS
TC36	length = 10	length = 10	PASS
TC37	All arrays not null	All arrays not null	PASS
TC38	luxuryDouble[0] not null	luxuryDouble[0] not null	PASS
TC39	luxurySingle[0] not null	luxurySingle[0] not null	PASS
TC40	deluxeSingle[2] = null	null	PASS
TC41	instanceof Exception = true	true	PASS
TC42	'Room Not Available'	'Room Not Available'	PASS

TC ID	Expected Output	Actual Output	Result
TC43	NotAvailable caught	NotAvailable caught	PASS
TC44	Not null, length > 0	Not null, length > 0	PASS
TC45	instanceof Runnable = true	true	PASS
TC46	No exception thrown	No exception thrown	PASS
TC47	Thread completes normally	Thread completes normally	PASS



4.4 Analysis of Results

On the initial run, all 119 test cases were successfully completed. The full pass rate indicates that the source code accurately implements the business logic components chosen for testing, including input validation, room data storage, billing computation, and room index mapping.

The Validator class's accuracy under all boundary circumstances and casing modifications is a crucial finding. Specifically, TC14 (phone prefix 013) and TC34–TC35 (mixed-case gender) verify that the implemented validation is both behaviorally robust and syntactically correct. The boundary tests for isValidPhone() (TC19 and TC20) verify that the regex applies a stringent yet suitable criterion for Bangladeshi mobile numbers: exactly 11 digits with no flexibility.

The function for `Hotel.getIndex()` appropriately handles all four different room ranges and returns -1 for each out-of-range input, according to the systematic boundary sweep. This is especially crucial since the conditional guards prevent the method's arithmetic formula (`index = roomNo - base`) from silently producing an array index that appears valid for inputs that are only marginally out of range.

In order for checkout to work properly for guests who did not order any food, the `getFoodBill()` tests verified that both room types handle the edge case of zero food orders appropriately (returning 0.0f rather than null or an error). The reliability of the float summation in a loop for the price magnitudes utilized in this system was confirmed by the floating-point addition test (TC22), which likewise passed without rounding errors.

The reflection-based testing of `getFoodBill()` additionally serves as indirect evidence that the `Food`. Since the prices supplied via reflection-set fields matched the expected values in all food-related tests, the `getPrice()` method is appropriately implemented.

5. Limitations of the Testing Work

The scope of this unit testing project did not allow for the coverage of several Hotel Management System components. These restrictions are explained below along with particular explanations.

The functions `Hotel.bookRoom()`, `Hotel.orderFood()`, `Hotel.checkout()`, and `Hotel.cancelBooking()` have not been tested. The fundamental transactional logic of the application—making reservations, adding food, calculating and showing invoices, and releasing rooms—is contained in these methods. All of these methods, however, read user input straight from a `Scanner` instance that is connected to `System.in`. Redirecting the system would be necessary for unit testing them, either changing the methods to accept input as parameters instead of reading from a global `Scanner`, or doing so at runtime (which is brittle and depends on the test environment). Within the parameters of this project, neither strategy was used.

In a similar vein, the `Food()` constructor is not directly checked. Prior to reading, the constructor prints a menu to `stdout` in addition to reading from `Scanner`. The actual price-calculation logic embedded in the `Food()` constructor's switch statement (which multiplies quantity by the per-item price) was not tested as a stand-alone unit, even though the reflection workaround used in `RoomTest.java` allows `Food` objects with known prices to be created for testing `getFoodBill()`. Extracting the price calculation into a different method or offering a constructor overload that takes pre-calculated values would be necessary for a specialized test.

Due to their reliance on the local filesystem, `FileSystem.save()` and `FileSystem.load()` were not included. The `hotel.dat` file is created and read by these methods in the working directory. Since they entail environment-dependent I/O side effects and require file system access to be available and writable, evaluating them would be considered integration testing rather than unit testing. Manual or system-level testing can confirm their accuracy.

The methods `Hotel.displayRoomDetails()` and `Hotel.displayAvailability()` return nothing and print formatted output to stdout. The normal JUnit `assertEquals` / `assertTrue` framework used in this project does not cover output testing such as capturing and asserting against stdout output, which is necessary to test them.

Lastly, no negative test cases with an improper type parameter (such as `type = 5`) were created for `Bill.calculate()`. According to the source code, if an invalid type is given, the method will return `0.0f` and both `roomCharge` and `foodCharge` will stay `0`. But instead of returning `0`, trying to call `h.luxuryDouble[num].getFoodBill()` on a null room object would result in a `NullPointerException`. `Bill.calculate()` did not specifically test this edge case.

6. Conclusion

The unit testing work done on a Java-based hotel management system is presented in this project report. A thorough test suite comprising 119 test cases was created and implemented using the JUnit 5 framework within the Eclipse IDE after the system was examined to determine which components could be subjected to automated unit testing. The `Validator` class (35 cases), the `Singleroom` and `Doubleroom` model classes (22 cases), the `Bill` calculation class (15 cases), and a combined utility test file covering `Hotel.getIndex()`, `Hotel.countAvailable()`, the holder data container, the `NotAvailable` exception, and the `write Runnable` (47 cases) were all tested.

The testing strategy combined a number of methods, including boundary value analysis for the hotel and positive and negative testing for the validator methods. `getIndex()` method, integration-style assertions for the `Bill` class, state verification for object constructors, and edge case testing for empty texts and lists. The Scanner-coupled `Food()` constructor presented a significant implementation problem that was resolved by utilizing Java Reflection and `sun.misc.Unsafe`. `Food` objects with controlled prices can be created using the `allocateInstance()` technique without calling the interactive constructor.

Upon initial execution, all 119 test cases were successful and error-free. The outcomes verify that `Validator`'s input validation mechanism is accurate and resilient to a variety of valid, invalid, and boundary inputs. The room model classes appropriately calculate food bills and store their fields. The `Bill.calculate()` method appropriately integrates with the meal bill and adds the appropriate

charge for each type of room. The `getIndex()` function protects against all out-of-range inputs and accurately translates room numbers to array indices. Although the application's business logic is good, a noteworthy finding from this testing method is that the system's testability is severely limited by the architectural coupling of business logic to Scanner input. Separating input processing from business logic would help future codebase enhancements by allowing for more thorough test coverage of the hotel's booking and checkout processes.